



# ค่ายโอลิมปิกวิชาการ วิชาคอมพิวเตอร์

---

## Library สำหรับการแข่งขัน

อาจารย์ ดร.อภิสิทธิ์ รัตนารักษ์

[apisit.r@sci.kmutnb.ac.th](mailto:apisit.r@sci.kmutnb.ac.th)

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ



# STL – Standard Template Library

---

- STL มี 3 components
  - Containers - class templates สำหรับเก็บข้อมูล
  - Algorithms - function templates สำหรับจัดการ Containers
  - Iterators - 'smart' pointers สามารถใช้งานกับ containers สร้างขึ้นเพื่อจัดการ STL Containers

# STL – Container

- Sequence Container
  - Vector
  - Deque
  - List
- Adapter Containers
  - Stack
  - Queue
  - Priority queue
- Associative Container
  - Set, multiset
  - Map, multimap

# STL – Standard Template Library

---

- STL มีการออกแบบที่มีประสิทธิภาพในแง่ของ Time complexity
- STL containers สามารถเพิ่มลดขนาดได้อัตโนมัติ
- STL มี built-in algorithms ในการจัดการ containers
- STL มี iterators ที่สามารถทำให้ containers และ Algorithms ทำงานได้อย่างยืดหยุ่นและมีประสิทธิภาพ
- STL สามารถสร้าง container และอัลกอริธึมได้
  - STL algorithms สามารถ process STL containers โดยใช้ user-defined containers ได้
  - User defined algorithms สามารถ process STL containers ได้

# Containers

Data structures ที่สามารถเก็บข้อมูล

- ☐ list: doubly linked list.
- ☐ vector: เหมือนกับ C array แต่ปรับค่าได้.
- ☐ map: set ของคู่ key/value
- ☐ Set: set ของ keys

# Algorithms

**generic** functions ที่ใช้ในงานทั่วไป ไม่ว่าจะเป็นการค้นหา การจัดเรียงข้อมูล การเปรียบเทียบ

- ☐ find

- ☐ merge

- ☐ reverse

- ☐ sort

- ☐ and more: count, random shuffle, remove, Nth-element, rotate.

# Vector

- มีโครงสร้างเช่นเดียวกับ array
- Vector สามารถเพิ่มขนาดลดขนาดได้.
- สามารถใช้แทน array ได้

# Defining Vector

Syntax: `vector<of what>`

เช่น:

`vector<int>` - vector of integers.

`vector<string>` - vector of strings.

`vector<int * >` - vector of pointers to integers.

`vector<Shape>` - vector of Shape objects. Shape เป็น class ที่สร้างขึ้นมาเอง.

# Using a Vector

- `#include <vector>`
- การใช้ vector type มี 2 วิธี:
  1. Array style.
  2. STL style

# Using a Vector (Array Style)

ใช้งานแบบ built-in array.

```
void simple_example()
{
    const int N = 10;
    vector<int> ivec(N);

    for (int i=0; i < 10; ++i)
        cin >> ivec[i];

    int ia[N];
    for ( int j = 0; j < N; ++j)
        ia[j] = ivec[j];
}
```



# Using a Vector (STL Style)

การสร้าง empty vector

```
vector<string> svec;
```

เราสามารถ insert ข้อมูลลง vector โดยใช้ method push\_back.

```
string word;
```

```
while ( cin >> word ) //the number of words is unlimited.
```

```
{
```

```
    svec.push_back(word);
```

```
}
```

# Insertion

```
void push_back(const T& x);
```

แทรก element x ตำแหน่งสุดท้าย

```
svec.push_back(str);
```

# size

```
unsigned int size();
```

Returns ความยาวของ vector

```
unsigned int size = svec.size();
```

# Problem

A horizontal red arrow pointing from the left towards the right, starting from the CIS logo and extending across the top of the slide.

เขียนโปรแกรมอ่านค่า **Integer** จาก **user** จากนั้นจัดเรียง และ  
แสดงผลลัพธ์

# STL

```
int main()
{
    int input;
    vector<int> ivec;

    /* rest of code */
}
```

# STL Input

```
while ( cin >> input )  
    ivec.push_back(input);
```

# STL Sorting

```
sort(ivec.begin(), ivec.end());
```

เมธอด sort

```
void sort(Iterator first, Iterator last);
```

# STL Output #1

---

```
for ( int i = 0; i < ivec.size(); ++i )  
    cout << ivec[i] << " ";  
cout << endl;
```

# STL Output #2

```
vector<int>::iterator it;  
for ( it = ivec.begin(); it != ivec.end(); ++it )  
    cout << *it << " ";  
cout << endl;
```

# STL include files

---

```
#include <iostream> // I/O
```

```
#include <vector>    // container
```

```
#include <algorithm> // sorting
```

```
//using namespace std;
```

# Putting it altogether

---

```
int main() {  
    int input;  
    vector<int> ivec;  
    // input  
    while (cin >> input )  
        ivec.push_back(input);  
    // sorting  
    sort(ivec.begin(), ivec.end());  
    // output  
    for (auto it = ivec.cbegin();  
         it != ivec.cend(); ++it ) {  
        cout << *it << " ";  
    }  
    cout << endl;  
    return 0;  
}
```

# Putting it altogether

```
for(auto riterator=x.crbegin();  
    riterator!=x.crend();  
    ++riterator)  
{  
    cout<<*riterator<<endl;  
}
```

# Operations on vector

- iterator [begin\(\)](#);
- iterator [end\(\)](#);
- bool [empty\(\)](#);
- void [push\\_back](#)(const T& x);
- iterator [erase](#)(iterator it);
- iterator [erase](#)(iterator first, iterator last);
- void [clear](#)();
- ....

# at()

```
int main() {  
    vector<int> ivec = {3,4,5,6,7};  
    cout<<ivec.at(3)<<endl;  
    ivec.at(3) = 10;  
    for(auto it=ivec.cbegin();it!=ivec.cend();++it){  
        cout<<*it<<endl;  
    }  
    return 0;  
}
```

# front() and back()

```
int main() {  
    int input;  
    vector<int> ivec = {3,4,5,6,7,8,9,10,11,12,13};  
  
    cout<<ivec.front()<<" "<<ivec.back()<<endl;  
  
    return 0;  
}
```

# insert()

```
int main() {  
    int input;  
    vector<int> ivec = {3,4,5,6,7,8,9,10,11,12,13};  
    ivec.insert(ivec.cbegin()+3,30);  
  
    for(auto it=ivec.cbegin();it!=ivec.cend();++it){  
        cout<<*it<<endl;  
    }  
    return 0;  
}
```

# erase()

```
vector<int> x = {3,4,5,6,7,3,4,5};  
x.erase(x.begin()+2);  
for (auto it = x.cbegin(); it!=x.cend(); ++it)  
{  
    cout<<*it<<" ";  
}  
x.erase(x.begin(),x.begin()+1);  
for (auto it = x.cbegin(); it!=x.cend(); ++it)  
{  
    cout<<*it<<" ";  
}
```

# list

- list เป็น sequence container จาก header list
- Class template list มีรูปแบบของ doubly linked list – ทุก Node มี pointer จาก previous node และ next node
- Member function
  - push\_front, pop\_front, ...
  - remove
  - unique
  - merge
  - reverse
  - sort
  - ...



# list - push\_front/push\_back

```
int main() {  
    list<int> val;  
    val.push_front(1);  
    val.push_front(2);  
    val.push_front(3);  
    val.push_back(4);  
    for(auto it=val.cbegin();it!=val.cend();it++){  
        cout<<*it<<" ";  
    }  
    return 0;  
}
```

# list - sort

```
int main() {  
    list<int> val;  
    val.push_front(1);  
    val.push_front(2);  
    val.push_front(3);  
    val.push_back(4);  
    val.sort();  
    for(auto it=val.cbegin();it!=val.cend();it++){  
        cout<<*it<<" ";  
    }  
    return 0;  
}
```

# list - merge

```
int main() {  
    list<int> val;  
    list<int> other = {2,4,6,8};  
    val.push_front(1);  
    val.push_front(2);  
    val.push_front(3);  
    val.push_back(4);  
    val.sort();  
    val.merge(other);  
    val.unique();  
    for(auto it=val.cbegin();it!=val.cend();it++){  
        cout<<*it<<" ";  
    }  
    return 0;  
}
```

// ทำให้ค่าภายในเป็นค่า unique เท่านั้น

# list - insert

```
int main() {  
    list<int> val;  
    list<int> other = {2,4,6,8};  
    val.push_front(1);  
    val.push_front(2);  
    val.push_front(3);  
    val.push_back(4);  
    val.insert(val.cbegin(),other.cbegin(),other.cend());  
    for(auto it=val.cbegin();it!=val.cend();it++){  
        cout<<*it<<" ";  
    }  
    return 0;  
}
```

# deque

- Double ended queue – สามารถ Push/Pop ได้ทั้ง head และ tail
- `#include<deque>`
- Member function
  - `push_front,`
  - `push_back,`
  - `pop_front,`
  - `pop_back`

# deque

```
int main() {  
    deque<int> deq;  
    deq.push_front(3);  
    deq.push_front(4);  
    deq.push_front(5);  
    deq.push_back(6);  
    for(auto it=deq.cbegin();it!=deq.cend();it++){  
        cout<<*it<<" ";  
    }  
    return 0;  
}
```

# deque

```
int main() {  
    deque<int> deq = {5,4,3,6};  
    for(auto it=deq.cbegin();it!=deq.cend();it++){  
        cout<<*it<<" ";  
    }  
    cout<<endl;  
    cout<<deq.front();                // Read ค่าออกมา  
    deq.pop_front();                  // Pop  
    cout<<endl;  
    for(auto it=deq.cbegin();it!=deq.cend();it++){  
        cout<<*it<<" ";  
    }  
    return 0;  
}
```

# stack & queue

---

- Stack – LIFO (`std::stack`)
- Queue – FIFO (`std::queue`)
- queue and stack เป็น container adaptors ถูก implemented บน `std::deque`.

# Next...

- Map
- pair
- copy algorithm

# Class Employee

```
class Employee {  
public:  
    // Constructors ...:  
    Employee () {}  
    Employee (const string& name) : _name(name) {}  
    // Member functions ...:  
    void set_salary(int salary) {_salary = salary; }  
    int salary() const { return _salary; }  
    void set_name(const string& name) { _name = name; }  
    const string& name() const { return _name;}  
    // ...  
private:  
    int _salary;  
    string _name;  
};
```

# Locating an Employee

---

เก็บข้อมูล employee ใน vector

หากต้องการหาข้อมูลให้ หาทีละคนใน vector

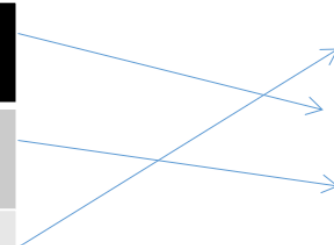
Bad solution - not efficient!

# Solution:

## Map - Associative Array

- ใช้ key/value pair มี index และ data เก็บข้อมูล
- `#include<map>`
- Insertion/find operation -  $O(\log n)$

|               |               |             |               |
|---------------|---------------|-------------|---------------|
| <b>A22-56</b> | <b>A24-57</b> | <b>8.75</b> | <b>Martin</b> |
| A23-57        | A22-56        | 12.50       | Calloway      |
| A24-57        | A23-57        | 4.95        | Mirage        |
|               |               |             |               |



# Map - make\_pair

```
int main() {  
  
    map<int,double,less<int>> pairs;  
  
    pairs.insert(make_pair(15,2.7));  
  
    pairs.insert(make_pair(30,111.11));  
  
    pairs.insert(make_pair(5,1010.1));  
  
    pairs.insert(make_pair(10,22.22));  
  
    pairs.insert(make_pair(25,33.333));  
  
    pairs.insert(make_pair(5,77.54));  
  
    pairs.insert(make_pair(20,9.345));  
  
    for(auto mapItem:pairs){  
  
        cout<<mapItem.first<< " "<<mapItem.second<<endl;  
  
    }  
    return 0;  
  
}
```

# Map - make\_pair

```
int main() {  
    map<int,double,less<int>> pairs;  
    pairs.insert(make_pair(15,2.7));  
    pairs.insert(make_pair(30,111.11));  
    pairs.insert(make_pair(5,1010.1));  
    pairs.insert(make_pair(10,22.22));  
    pairs.insert(make_pair(25,33.333));  
    pairs.insert(make_pair(5,77.54));  
    pairs.insert(make_pair(20,9.345));  
    pairs[25] = 9999.99;  
    pairs[40] = 8765.43;  
    for(auto mapItem:pairs){  
        cout<<mapItem.first<< " "<<mapItem.second<<endl;  
    }  
    return 0;  
}
```

# Solution:

## Map - Associative Array

ถ้าใช้ map key เป็นชื่อ employee และ value เป็น employee object.

name           ->           employee.

string          ->           class Employee

```
map<string, Employee *> employees;
```

# Populating a Map

```
void main()
{
    map<string, Employee *> employees;

    string name("Eti");
    Employee *employee;

    employee = new Employee(name);

    //insertion
    employees[name] = employee;
}
```

# Locating an Employee

```
map<string, Employee> employees;
```

หาชื่อ employee Eti:

```
//find
```

```
Employee *eti = employees[“Eti”];
```

```
//or
```

```
map<string, Employee *>::iterator iter = employees.find(“Eti”);
```

The returned value is an iterator to map.

ถ้า “Eti” มีใน map iterator จะชี้ไปที่วัตถุนั้น ถ้าไม่มีจะ return end() iterator ของ map.

# Iterating Across a Map

แสดงข้อมูลของ map ทั้งหมด

```
map<string,Employee *>::iterator it;
for ( it = employees.begin();
      it != employees.end(); ++it )
{
    cout << ???
}
```

# map vs multimap

```
int main() {  
    multimap<int,double,less<int>> pairs;  
    pairs.insert(make_pair(15,2.7));  
    pairs.insert(make_pair(30,111.11));  
    pairs.insert(make_pair(5,1010.1));  
    pairs.insert(make_pair(10,22.22));  
    pairs.insert(make_pair(25,33.333));  
    pairs.insert(make_pair(5,77.54));  
    pairs.insert(make_pair(20,9.345));  
    for(auto mapItem:pairs){  
        cout<<mapItem.first<< " "<<mapItem.second<<endl;  
    }  
    return 0;  
}
```

# Iterators

เป็นวิธีการทั่วไปในการเข้าถึงแต่ละค่าของ vector, list ที่มีรูปแบบเป็น linear หรือ associative เช่น map, set containers

# Pointer Semantics

หาก iter เป็น iterator เราสามารถใช้งานได้ดังนี้ :

- ++iter (or iter++)  
ขยับ iterator ไปตำแหน่งถัดไป
- \*iter Returns ค่าของ element นั้นออกมา

# Begin & End

แต่ละ container สามารถใช้ ฟังก์ชัน `begin()` และ `end()`

- `begin()` Returns iterator ตำแหน่งแรกของ container.
- `end()` returns ตำแหน่งสุดท้าย

# Iterating Over Containers

Iterating ไปในแต่ละ elements ของ container type.

```
for ( iter = container.begin();  
    iter != container.end();  
    ++iter )  
{  
    // do something with the element  
}
```

# Map Iterators

`map<key, value>::iterator iter;`

Type ของ element iter จะใช้ key หรือ value?

จะใช้ key/value pair.

# Pair

สามารถเก็บ pair ของ objects, type แรกคือ  $T_1$  และ type สองคือ  $T_2$ .

```
struct pair<T1, T2>  
{  
    T1 first;  
    T2 second;  
};
```

# Pair

ในตัวอย่าง iter คือ

```
pair<string, Employee *>
```

Element.

เข้าถึง name (key) โดย

```
iter->first
```

เข้าถึง Employee\* (value) โดย

```
iter->second
```

# Printing Salary

```
for ( iter = employees.begin() ;  
      iter != employees.end() ;  
      ++iter )  
{  
    cout << iter->first << " "  
          << (iter->second)->salary() ;  
}
```

# Copy

`copy(iterator first, iterator last, iterator where);`

Copy ตั้งแต่ 'first' ถึง 'last' ไปยัง 'where'.

```
int ia[] = { 0, 1, 1, 2, 3, 5, 5, 8 };  
vector<int> ivec1(ia, ia + 8 ), ivec2;
```

```
// ...
```

```
copy(ivec1.begin(), ivec1.end(),  
      back_inserter(ivec2));
```

# Algorithms

- `#include<algorithm>`
- Sort
- Search
- Count

# sort

```
int main() {  
    vector<int> x = {20,50,80,90,100,15};  
    sort(x.begin(),x.end());  
    for(auto it=x.cbegin();it!=x.cend();it++){  
        cout<<*it<<" ";  
    }  
    return 0;  
}
```

# find

```
#include<iterator> //*****  
  
int main() {  
    vector<int> x = {20,50,80,90,100,15};  
    vector<int>::iterator it;  
    it = find(x.begin(),x.end(),90);  
    if(it!=x.end())  
        cout<<"Found at "<<it-x.begin()<<" "<<endl;;  
    else  
        cout<<"Not found"<<endl;  
    return 0;  
}
```

# find\_if

```
bool gt10(int value){
    return value>10;
}

int main() {
    vector<int> x = {20,50,80,90,100,15};
    vector<int>::iterator it;
    it = find_if(x.begin(),x.end(),gt10);
    if(it!=x.end())
        cout<<"Found greater than 10 at "<<it-x.begin()<<" "<<endl;;
    else
        cout<<"Not found"<<endl;
    return 0;
}
```

# all\_of

```
bool gt10(int value){
    return value>10;
}

int main() {
    vector<int> x = {20,50,80,90,100,15};
    bool it = all_of(x.begin(),x.end(),gt10);
    if(it)
        cout<<"All elements are greater than 10"<<endl;

    return 0;
}
```

# any\_of

```
bool gt10(int value){
    return value>10;
}

int main() {
    vector<int> x = {20,50,80,90,100,15};
    bool it = any_of(x.begin(),x.end(),gt10);
    if(it)
        cout<<"Some elements are greater than 10"<<endl;

    return 0;
}
```

# none\_of

```
bool gt10(int value){
    return value>10;
}

int main() {
    vector<int> x = {20,50,80,90,100,15};
    bool it = none_of(x.begin(),x.end(),gt10);
    if(it)
        cout<<"None elements are greater than 10"<<endl;

    return 0;
}
```

# count

```
int main() {  
    vector<int> x = {20,50,80,90,100,15,100,100};  
    int res = count(x.begin(),x.end(),100);  
    cout<<res<<endl;  
    return 0;  
}
```

# count

```
bool gt10(int value){
    return value>10;
}

int main() {
    vector<int> x = {20,50,80,90,100,15,100,100};
    int res = count_if(x.begin(),x.end(),gt10);
    cout<<res<<endl;
    return 0;
}
```

# min\_element & max\_element

---

```
int main() {  
    vector<int> x = {20,50,80,90,100,15,100,100};  
    vector<int>::iterator it;  
    it = min_element(x.begin(),x.end());  
    cout<<it-x.begin()<<endl;  
    it = max_element(x.begin(),x.end());  
    cout<<it-x.begin()<<endl;  
    return 0;  
}
```

# accumulate

```
#include<numeric> //*****  
  
int main() {  
    vector<int> x = {1,2,3,4,5,6,7,8,9,10};  
    cout<<accumulate(x.cbegin(),x.cend(),0);  
    return 0;  
}
```

# for\_each

```
void calsquare(int val){  
    cout<<val*val<<" ";  
}  
  
int main() {  
    vector<int> x = {20,50,80,90,100,15,100,100};  
  
    for_each(x.cbegin(),x.cend(),calsquare);  
    return 0;  
}
```

# reverse

```
int main() {  
    vector<int> x = {1,3,5,6,7,9};  
    reverse(x.begin(),x.end());  
    for(auto it=x.cbegin();it!=x.cend();it++){  
        cout<<*it<<" ";  
    }  
    return 0;  
}
```

# reverse

```
int main() {  
    vector<int> x = {1,3,5,6,7,9};  
    reverse(x.begin(),x.end());  
    for(auto it=x.cbegin();it!=x.cend();it++){  
        cout<<*it<<" ";  
    }  
    return 0;  
}
```

# partition

```
bool func(int x){  
    if(x<5)  
        return true;  
    else  
        return false;  
}
```

```
int main(){  
  
    vector<int> x = {3,4,5,6,7,3,4,5};  
    vector<int>::iterator bound;  
    bound = partition(x.begin(),x.end(),func);  
    for(auto it=x.cbegin();it!=bound;it++)  
        cout<<*it<<" ";  
  
    cout<<endl;  
    for(auto it=bound;it!=x.cend();it++)  
        cout<<*it<<" ";  
  
    return 0;  
}
```

# make\_heap & sort\_heap

```
int main(){  
    vector<int> x = {1,3,5,7,9};  
    vector<int>::iterator it;  
    make_heap(x.begin(),x.end());  
    for(it=x.begin();it!=x.end();it++)  
        cout<<*it<<" ";  
    cout<<endl;  
    sort_heap(x.begin(),x.end());  
    for(it=x.begin();it!=x.end();it++)  
        cout<<*it<<" ";  
    cout<<endl;  
}
```

# merge

```
int main(){  
    vector<int> x = {9,5,3,7,1};  
    vector<int> y = {8,6,2,4,10};  
    vector<int> z;  
    vector<int>::iterator it;  
    sort(x.begin(),x.end());  
    sort(y.begin(),y.end());  
    merge(x.begin(),x.end(),y.begin(),y.end(),back_inserter(z));  
    for(it=z.begin();it!=z.end();it++)  
        cout<<*it<<" ";  
    cout<<endl;  
    return 0;  
}
```

- Container แบ่งออกเป็น Sequence, Adapter และ Associative Containers (แนะนำให้ใช้ Sequence และ Associative)
- STL Library มี algorithm ให้เลือกใช้ ให้เหมาะสมกับโปรแกรมที่จะเขียน
- หากสามารถเรียก library ได้ ให้เลือกใช้ library ก่อนเพื่อประหยัดเวลาในการเขียนโปรแกรม
- มีอีกหลายฟังก์ชันของ STL ที่สามารถใช้งานได้ที่ไม่ได้แนะนำในวันนี้